Case Study: Simple Employee Management System

Build a Node.js app with MySQL to manage employees.

Features:

1. Add new employees (name, email, department).

2. List all employees.

3. Update employee information.

4. Delete an employee.

This teaches:

• Connecting Node.js to MySQL

• CRUD operations (Create, Read, Update, Delete)

• Using parameterized queries to avoid SQL injection

Step 1: Setup MySQL Database

1. Open MySQL Workbench or CLI.

2. Create a new database:

```
CREATE DATABASE employeeDB;

USE employeeDB;
```

3. Create a table employees:

```
CREATE TABLE employees (

id INT AUTO_INCREMENT PRIMARY KEY,

name VARCHAR(100) NOT NULL,

email VARCHAR(100) NOT NULL UNIQUE,

department VARCHAR(50),

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

);
```

Step 2: Project Setup

```
mkdir employee-management

cd employee-management
```

npm init -y

npm i mysql2

Step 3: Connect Node.js to MySQL

Step 4: Insert Employee

Step 5: Fetch All Employees

Step 6: Update Employee Info

Step 7: Delete Employee

Step 8: Close Connection

Step 9: Full Workflow

1. Connect to MySQL
2. Insert employees
3. Fetch all employees
4. Update an employee
5. Delete an employee
6. Close connection

```
const mysql = require('mysql2/promise');

async function main() {
  const conn = await mysql.createConnection({
    host: 'localhost',
    user: 'root',
    password: 'root',
    database: 'employeeDB',
  });
  console.log('MySQL connected');

  async function addEmployee(name, email, department) {
    const [res] = await conn.execute(
      'INSERT INTO employees (name, email, department) VALUES (?, ?, ?)',
      [name, email, department]
```

```
    );
    return res.insertId;
  }

  async function listEmployees() {
    const [rows] = await conn.execute('SELECT * FROM employees ORDER BY id');
    return rows;
  }



  async function updateEmployee(id, fields) {
    const keys = Object.keys(fields);
    if (keys.length === 0) return;
    const sets = keys.map(k => `${k} = ?`).join(', ');
    const values = keys.map(k => fields[k]);
    values.push(id);
    await conn.execute(`UPDATE employees SET ${sets} WHERE id = ?`, values);
  }
  async function deleteEmployee(id) {
    await conn.execute('DELETE FROM employees WHERE id = ?', [id]);
  }
  await conn.execute('DELETE FROM employees');
  const aliceId = await addEmployee('Alice', 'alice@example.com', 'Engineering');
  await addEmployee('Bob', 'bob@example.com', 'HR');
  console.log('All employees:', await listEmployees());
  await updateEmployee(aliceId, { department: 'R&D', email: 'alice@company.com' });
  console.log('After update:', await listEmployees());
  await deleteEmployee(aliceId);
  console.log('After delete:', await listEmployees());
  await conn.end();
  console.log('MySQL disconnected');
}
main().catch(err => { console.error(err); process.exit(1); });
```